Projec :- Loan Approval Prediction using Logistic Regression

Problem Statement

Financial institutions need to assess whether a loan applicant is eligible for approval based on various factors such as income, credit history, employment status, and demographics.

The challenge is to build a machine learning model that can predict whether a loan will be approved or not, helping banks reduce risk and automate decision-making.

Objective

- Analyze applicant data to identify factors influencing loan approval
- Handle missing values using proper preprocessing techniques
- Encode categorical features and scale numerical features
- Build a Logistic Regression model using Pipeline
- Evaluate model using classification metrics
- Generate meaningful business insights

STEP 1: LOAD + CLEAN DATA

In [1]:
```
import pandas as pd

df = pd.read_csv("Loan_dataset.csv")

# drop unnecessary columns
df = df.drop(["Unnamed: 0", "Loan_ID"], axis=1)

print(df.head())
print(df.info())
```

```
   Gender Married Dependents     Education Self_Employed  ApplicantIncome  \
0  Female      No          0      Graduate            No             4547
1    Male     Yes         3+  Not Graduate           Yes             5703
2  Female     Yes          0      Graduate            No             4333
3    Male     Yes          0  Not Graduate           Yes             4695
4    Male     Yes          2      Graduate            No             6700

   CoapplicantIncome  LoanAmount  Loan_Amount_Term  Credit_History  \
0                0.0       115.0             360.0             1.0
1                0.0       130.0             360.0             1.0
2             2451.0       110.0             360.0             1.0
3                0.0        96.0               NaN             1.0
4             1750.0       230.0             300.0             1.0

  Property_Area  Loan_Status
0     Semiurban            1
1         Rural            1
2         Urban            0
3         Urban            1
4     Semiurban            1
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 491 entries, 0 to 490
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Gender             481 non-null    object
 1   Married            490 non-null    object
 2   Dependents         482 non-null    object
 3   Education          491 non-null    object
 4   Self_Employed      462 non-null    object
 5   ApplicantIncome    491 non-null    int64
 6   CoapplicantIncome  491 non-null    float64
 7   LoanAmount         475 non-null    float64
 8   Loan_Amount_Term   478 non-null    float64
 9   Credit_History     448 non-null    float64
 10  Property_Area      491 non-null    object
 11  Loan_Status        491 non-null    int64
dtypes: float64(4), int64(2), object(6)
memory usage: 46.2+ KB
None
```

- Check Missing values

In [2]:
```
print(df.isnull().sum())
```

```
Gender               10
Married               1
Dependents            9
Education             0
Self_Employed        29
ApplicantIncome       0
CoapplicantIncome     0
LoanAmount           16
Loan_Amount_Term     13
Credit_History       43
Property_Area         0
Loan_Status           0
dtype: int64
```

STEP 2: DEFINE X and y

In [3]:
```
X = df.drop("Loan_Status", axis=1)
y = df["Loan_Status"]
```

STEP 3: TRAIN-TEST SPLIT

In [4]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2, random_s
```

STEP 4: FULL PREPROCESSING PIPELINE

Step 4.1: Import Libraries

In [5]:
```
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

Step 4.2: Separate Columns

In [6]:
```
cat_cols = [
    "Gender", "Married", "Dependents",
    "Education", "Self_Employed", "Property_Area"
]

num_cols = [
    "ApplicantIncome", "CoapplicantIncome",
    "LoanAmount", "Loan_Amount_Term", "Credit_History"
]
```

Step 4.3: Create Pipelines

- Numerical Pipeline

In [7]:
```
num_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("scalar", StandardScaler() )
])
```

- Categorical Pipeline

In [8]:
```
cat_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(drop="first"))
])
```

Step 4.4: Combine using ColumnTransformer

In [9]:
```
preprocessor = ColumnTransformer([
    ("num", num_pipeline, num_cols),
    ("cat",cat_pipeline, cat_cols)
])
```

Why SimpleImputer?

- Handles missing values inside pipeline Prevents data leakage

Why separate pipelines?

- Different preprocessing for different data types

STEP 5: BUILD MODEL PIPELINE

Step 5.1 Import Logistic Regression

In [10]:
```
from sklearn.linear_model import LogisticRegression
```

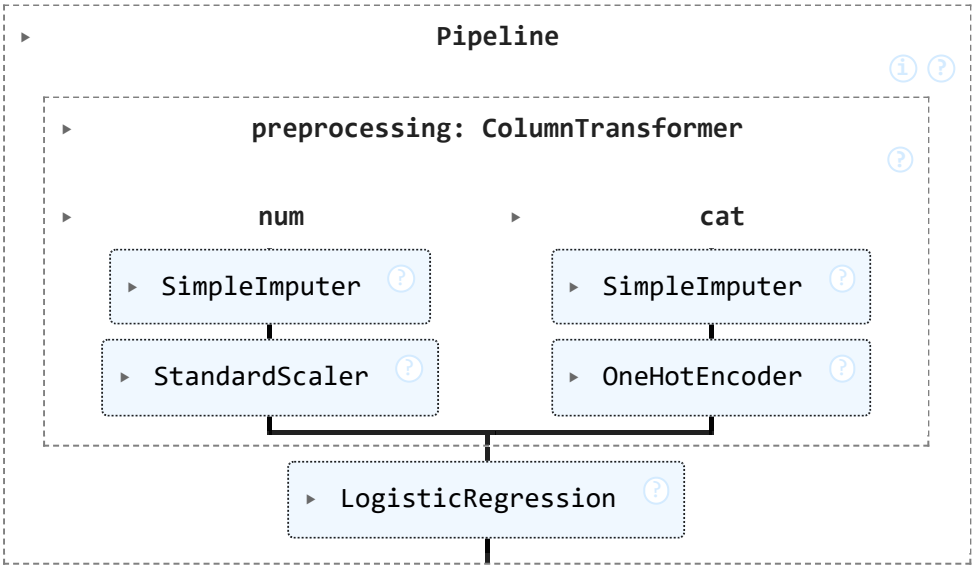
Step 5.2 Create Full Pipeline

In [11]:
```
model = Pipeline([
    ("preprocessing", preprocessor),
    ("classification", LogisticRegression())
])
```

STEP 6: TRAIN MODEL

In [12]:
```
model.fit(X_train, y_train)
```

Out[12]:



STEP 7: PREDICTIONS

In [13]:
```
y_pred = model.predict(X_test)
```

STEP 8: EVALUATION

In [14]:
```python
from sklearn.metrics import accuracy_score, confusion_matrix, classification_rep

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.8282828282828283

Confusion Matrix:
 [[17 17]
 [ 0 65]]

Classification Report:
               precision    recall  f1-score   support

           0       1.00      0.50      0.67        34
           1       0.79      1.00      0.88        65

    accuracy                           0.83        99
   macro avg       0.90      0.75      0.78        99
weighted avg       0.86      0.83      0.81        99
```

INSIGHTS

- Credit History is likely the strongest factor for loan approval
- Applicants with stable income have higher approval chances
- Model favors predicting approvals over rejections
- Class imbalance or feature influence may affect predictions
- Improving rejection detection is critical for business use

FINAL CONCLUSION

- The Logistic Regression model achieved an accuracy of 82.8% and performed well in identifying approved loans. However, it showed limitations in detecting rejected applications, which is critical in financial decision-making. Further improvements such as class balancing or threshold tuning can enhance model performance.